

Experiment No: 1

Date of Performance :-

Title: Implement a simple linear regressor with a single/multilayer neuron model.

Aim: - To implement a Simple Linear Regression model that fits the given data and makes predictions for the output variable based on inputs.

Theory:

Linear regression is one of the easiest and most popular Machine Learning algorithms. It is a statistical method that is used for predictive analysis. Linear regression makes predictions for continuous/real or numeric variables such as sales, salary, age, product price, etc.

Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (x) variables, hence called as linear regression. Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.

The linear regression model provides a sloped straight line representing the relationship between the variables.

Connection to a single neuron: A linear regression model is mathematically identical to a single artificial neuron with a linear (identity) activation function. The neuron computes a weighted sum of its inputs plus a bias, $y = w \cdot x + b$, where w is the weight (slope) and b is the bias (intercept). Training the model means finding the values of w and b that best fit the data — the same idea used later for neurons with non-linear activations in larger neural networks.

How the best-fit line is found: The model's error is measured using the Mean Squared Error (MSE) cost function, which averages the squared difference between predicted and actual values. Training adjusts w and b to minimise this cost — either through the closed-form solution used internally by Sklearn's Linear Regression, or iteratively via gradient descent.

Types of Linear Regression:

Linear regression can be further divided into two types of the algorithm:

1. **Simple Linear Regression:** If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.
 - Uses one independent variable.
 - Example: Predicting salary based on experience.
2. **Multiple Linear Regression:** If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.
 - Uses two or more independent variables.
 - Example: Predicting house price based on size, location, and number of rooms.

Tools & Dataset:

- Python Libraries Used: pandas, matplotlib, Sklearn
- **Dataset:** simpleio1.xlsx / Simpleio.xlsx (both refer to the same simple X–Y sample data — use whichever copy is available) and California.csv.
 - simpleio1.xlsx / Simpleio.xlsx: two columns — input (X) and output (Y) — used for the Simple Linear Regression part (Part A).
 - California.csv: multiple feature columns (e.g., median income, house age, average rooms, population, location) plus a target column (median house value) — used for the Multiple Linear Regression part (Part B).

Program Code:**Part A) Performing regression over simple input and output using simpleio data files.**

1. Import pandas, matplotlib.pyplot, and Linear Regression from sklearn.linear_model.
2. Load simpleio1.xlsx / Simpleio.xlsx into a DataFrame using pandas.read_excel().
3. Separate the data into input X and output Y (reshape X to 2D if needed for sklearn).
4. Split the data into training and testing sets using train_test_split().
5. Create a Linear Regression() model and fit it on the training data.
6. Predict Y values for the test set and print the learned slope (coef_) and intercept (intercept_).
7. Plot the actual data points as a scatter plot and overlay the predicted regression line.

Part B) Predicting house price. Using California.csv.

1. Load California.csv into a DataFrame using pandas.read_csv().
2. Select the feature columns as X and the target (house value) column as Y.
3. Split the data into training and testing sets.
4. Fit a LinearRegression() model (this is now Multiple Linear Regression, since there are several features).
5. Predict house prices for the test set and compare predicted vs. actual values.
6. Evaluate the model using metrics such as R^2 score and Mean Squared Error (from sklearn.metrics).

(Students are encouraged to use any data from any cities)

Output of the Program:

- PART A:**

```
import pandas as pd

df = pd.read_csv(r"C:\Users\sudar\Documents\Collage\Placement prep\Machine learning and computer vision\Lab 2\Simpleio.csv")
df=df.values
xs=df[:,0]
ys=df[:,1]
```

[2]

```
X_train=xs[0:70]
X_test=xs[71:]
Y_train=ys[0:70]
Y_test=ys[71:]
print(X_train)
print(X_test)
print(Y_train)
print(Y_test)
```

[3]

```
... [0.04981121 0.09962241 0.1549682  0.21031398 0.26565976 0.32100555
0.38250086 0.44399618 0.51232431 0.58065244 0.65657258 0.74092829
0.83465687 0.93879972 1.06737115 1.21022829 1.36895845 1.5453253
1.70405545 1.8469126  1.98976974 2.13262688 2.27548403 2.41834117
2.56119831 2.70405545 2.8469126  2.98976974 3.13262688 3.27548403
3.43421418 3.59294434 3.76931119 3.94567803 4.12204487 4.29841172
4.47477856 4.6511454  4.82751224 4.9862424  5.14497256 5.30370272
5.44655986 5.58941701 5.73227415 5.89100431 6.06737115 6.24373799
6.38659514 6.52945228 6.64516657 6.73889514 6.82325085 6.89917099
6.96749912 7.02899444 7.09048975 7.14583554 7.20118132 7.25652711
7.31187289 7.3616841  7.4114953  7.46130651 7.51665229 7.57199808
7.62734386 7.68268964 7.74418496 7.80568028]
[7.94992855 8.03428426 8.13842712 8.28128426 8.4576511  8.60050825
8.71622253 8.8099511  8.89430682 8.97022696 9.0461471  9.11447523
9.18280336 9.25113149 9.31945962 9.38778775 9.46370789 9.53962803
9.62398375 9.71771232 9.8334266  9.97628375]
[ 5.3578233  5.66317734  5.99546497  6.31947144  6.63426069  6.93891889
 7.2644727  7.57527423  7.90199367  8.2078077  8.52162186  8.83659809
 9.14322459 9.42886211  9.70066606  9.89952808 10.         9.97857467
 0.0500540  0.50755306  0.47331040  0.33033703  0.07000030  0.70000030]
```

The **Simpleio.csv** dataset was loaded using Pandas. The dataset contains two columns, where the first column is used as the input variable X and the second column is used as the output variable Y. The data was converted into a NumPy array and separated into input and output variables for regression analysis. The dataset was divided into training and testing sets. The first **70 samples** were used for **training**, while the remaining samples were used for testing.

```
model.summary()
```

[6]

```
... Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 1)	2
dense_1 (Dense)	(None, 10)	20
dense_2 (Dense)	(None, 1)	11

```
... Total params: 33 (132.00 B)
... Trainable params: 33 (132.00 B)
... Non-trainable params: 0 (0.00 B)
```

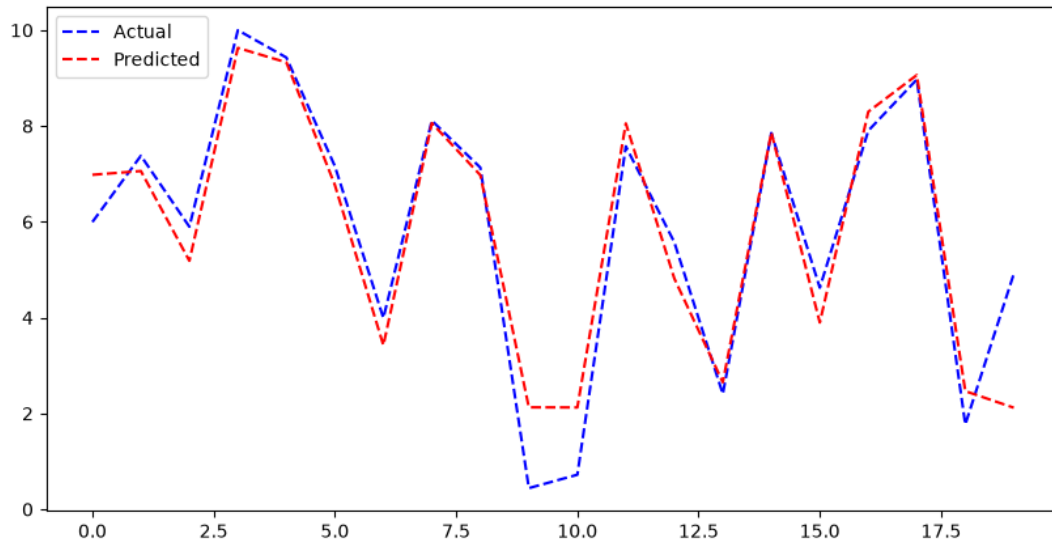
A multilayer neural network was implemented for univariate regression. The model consists of an input layer with one neuron, followed by a **hidden Dense layer containing 10 neurons with sigmoid activation**, and a final output layer containing one neuron for predicting the continuous output value. The network learns the relationship between the input variable and output variable by adjusting its weights and biases during training.

```
model.evaluate(X_test,Y_test,verbose=2)
```

[11]

```
... 1/1 - 0s - 80ms/step - loss: 0.8423 - mean_squared_error: 0.8423
... [0.8423276543617249, 0.8423276543617249]
```

The trained neural network was evaluated using the test dataset. The model achieved a Mean Squared Error (MSE) of **0.8423** on the test data. The test loss is also **0.8423**, since MSE was used as the loss function.



The graph compares the actual output values with the values predicted by the trained neural-network regression model for the test dataset. The predicted values generally follow the trend of the actual values. The differences between the two lines represent the prediction errors of the model.

PART B:

```
housing=pd.read_csv(r"C:\Users\sudar\Documents\Collage\Placement prep\Machine learning and computer vision\Lab 2\housing.csv")

#display entire data frame, few rows and columns, top 5 and bottom 5
housing
```

[2]

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY

The California Housing dataset was loaded from the housing.csv file using the Pandas library. The dataset contains geographical, demographic, and housing-related attributes such as longitude, latitude, housing median age, total rooms, total bedrooms, population, households, and median income. The target variable used for prediction is median_house_value. The dataset also contains the categorical feature ocean_proximity.

```
housing.pop('ocean_proximity')
housing.pop('total_bedrooms')
housing
```

[3]

	longitude	latitude	housing_median_age	total_rooms	population	households	median_income	median_house_value
0	-122.23	37.88	41.0	880.0	322.0	126.0	8.3252	452600.0
1	-122.22	37.86	21.0	7099.0	2401.0	1138.0	8.3014	358500.0
2	-122.24	37.85	52.0	1467.0	496.0	177.0	7.2574	352100.0
3	-122.25	37.85	52.0	1274.0	558.0	219.0	5.6431	341300.0
4	-122.25	37.85	52.0	1627.0	565.0	259.0	3.8462	342200.0
...	...	...	...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	845.0	330.0	1.5603	78100.0
20636	-121.21	39.49	18.0	697.0	356.0	114.0	2.5568	77100.0
20637	-121.22	39.43	17.0	2254.0	1007.0	433.0	1.7000	92300.0
20638	-121.32	39.43	18.0	1860.0	741.0	349.0	1.8672	84700.0
20639	-121.24	39.37	16.0	2785.0	1387.0	530.0	2.3886	89400.0

The `ocean_proximity` and `total_bedrooms` columns were removed from the California Housing dataset using the `pop()` function. The categorical `ocean_proximity` feature was removed to retain numerical input features for the neural-network model. After removing these columns, the dataset contains 20,640 rows and 8 columns, including seven input features and the target variable `median_house_value`.

```
model = keras.models.Sequential([ keras.layers.Dense(14, activation="relu", input_shape=(7,)),
                                keras.layers.Dense(1)])
```

[17]

```
... c:\Users\sudar\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\src\layers\core\dense.py:107:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.summary()
```

[18]

```
... Model: "sequential"
```

...

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 14)	112
dense_1 (Dense)	(None, 1)	15

...

```
Total params: 127 (508.00 B)
```

...

```
Trainable params: 127 (508.00 B)
```

...

```
Non-trainable params: 0 (0.00 B)
```

The multivariate regression model consists of an input layer with **7 features**, a hidden Dense layer containing **14 neurons with ReLU activation**, and an output Dense layer containing **1 neuron**. The model has a total of **127 trainable parameters**. The model was compiled using **Mean Squared Error (MSE)** as the loss function and **Stochastic Gradient Descent (SGD)** as the optimizer. It was trained for **50 epochs**.

```
484/484 ————— 0s 643us/step - loss: 53572.5430
```

```
Epoch 12/50
```

```
484/484 ————— 0s 600us/step - loss: 53379.0469
```

...

```
Epoch 49/50
```

```
484/484 ————— 0s 590us/step - loss: 47521.9648
```

```
Epoch 50/50
```

```
484/484 ————— 0s 585us/step - loss: 47545.8438
```

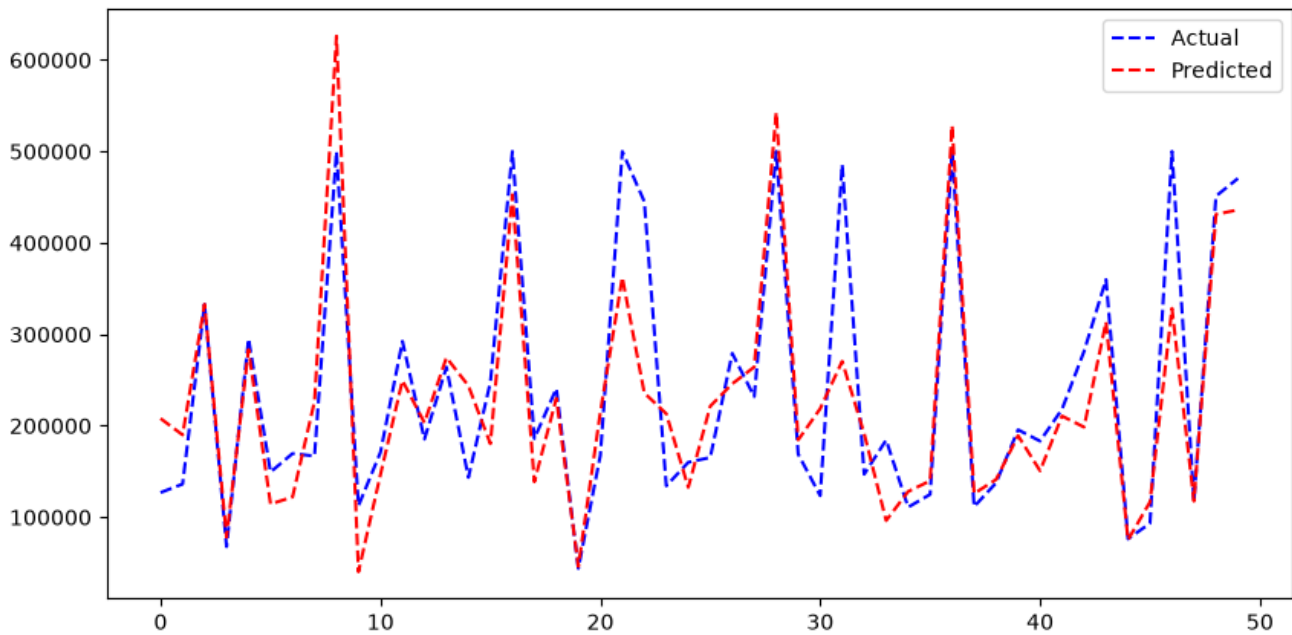
Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

```
htest = model.evaluate(X_test, y_test)
```

]

```
162/162 ————— 0s 646us/step - loss: 47163.5117
```

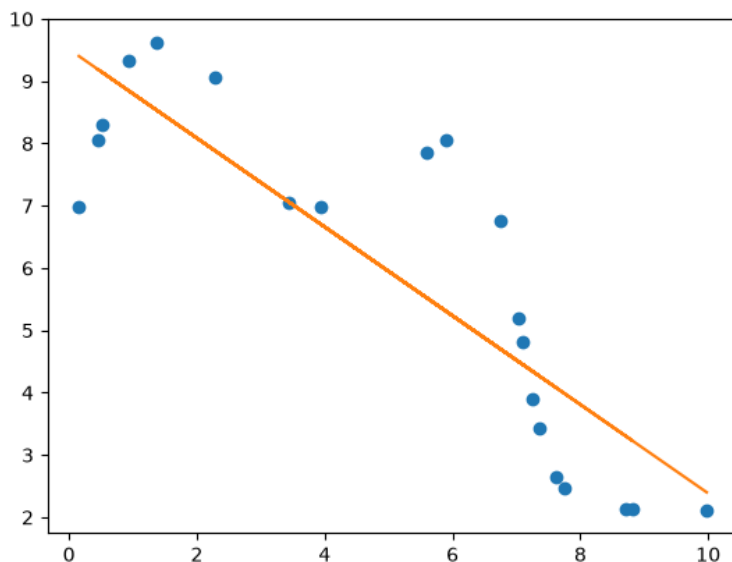
The trained multivariate regression model was evaluated using the test dataset after 50 epochs of training. The model uses Mean Squared Error (MSE) as the loss function and Stochastic Gradient Descent (SGD) as the optimizer. The obtained test MSE was 47163.5117.



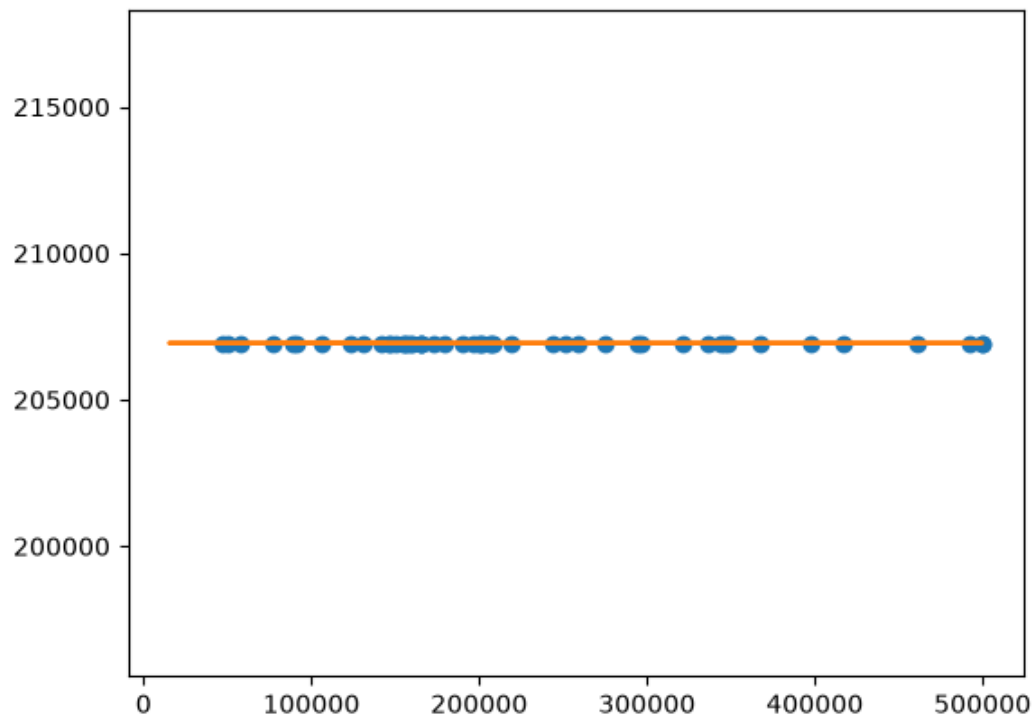
The graph compares the actual median house values with the values predicted by the multivariate neural-network regression model for 50 test samples. The blue dashed line represents the actual values, while the red dashed line represents the predicted values. The predicted values remain close to approximately the same level, whereas the actual values show considerable variation. This indicates that the model is not capturing the variations in the target values effectively and therefore has relatively high prediction error.

Regression Plot Explanation:

PART A:



The scatter plot shows the relationship between the input variable (X) and output variable (Y), along with the fitted regression line. The regression line has a negative slope, indicating that the predicted output generally decreases as the input value increases. The intercept represents the estimated output value when the input is zero. The data points show a noticeable spread around the fitted line, indicating that the linear model does not perfectly fit all the observations. Some points are relatively far from the regression line, suggesting higher prediction error or possible outliers. Overall, the plot indicates a negative linear relationship, but the fit is not very strong because several data points deviate considerably from the fitted line.

PART B:

The scatter plot shows the relationship between the input/target values and the predicted values along with the fitted regression line. The predicted values are concentrated around a narrow range near 209,000, while the actual data has a much wider range. This indicates that the model is producing nearly constant predictions and is not effectively capturing the variation in the target variable.

Conclusion:

The experiment successfully demonstrated **univariate and multivariate regression using neural-network models**. In Part A, the model learned the relationship between the input and output variables and achieved a **test MSE of 0.8423**. In Part B, a multivariate regression model using seven input features was implemented and achieved a **test MSE of 47,163.5117**. The Actual vs. Predicted plots showed that the Part B model did not capture the variation in house values effectively. The performance can be improved through better preprocessing, feature selection, hyperparameter tuning, suitable network architecture, and appropriate training.